KOI CONTENT EXTENSION

Test Guide

Ten tests for the content built on the Marketplace KOI integration — with the setup each one needs

For anyone comfortable in the Cortex XSIAM console. No content development required.

Extension 1.3.0 · requires the Marketplace KOI pack · July 2026

CONTENTS

What ships in this extension

A	**Parsing & modeling rules**	Normalise KOI events and map them to the Cortex Data Model. The Marketplace pack ships none.
B	**Alerts dashboard**	Ready-made view of KOI alert activity.
C	**Alert fields, type & layout**	19 KOI fields written onto each alert and shown in a purpose-built layout.
D	**Triage & investigation playbooks**	6 playbooks: triage, item and device investigation, enrichment, gated response.
E	**Hunting playbooks**	3 playbooks: MCP server audit, hunt sweep, and its investigation sub-playbook.
F	**Script Runner playbooks**	5 playbooks plus the KoiScanTracker automation — run KOI scripts fleet-wide.

The integration itself is NOT in this pack — it comes from the Marketplace KOI pack, which must be installed first.

HOW TO USE THIS GUIDE

Every test is laid out the same way

WHAT YOU NEED FIRST

The tenant setup this test depends on. If something here is missing the test cannot pass, and it will look like a product fault when it is not.

STEPS

What to click, in order, in the Cortex XSIAM console. Anything to paste is shown in monospace.

WHAT TO EXPECT

What a passing test looks like. Where a result commonly looks wrong but is correct, it says so.

Run them in order the first time

Tests 1 to 3 prove data is arriving and correct. Tests 4 to 7 prove the automation. Tests 8 and 9 prove fleet script execution — the only ones needing the Core REST API integration. Test 10 is the alert layout. Nothing here needs a Cortex XDR integration: XSIAM is the XDR, and its XQL engine is built in.

A test that fails on a missing prerequisite is not a product defect — check the amber panel first.

BEFORE YOU BEGIN

Everything this pack depends on, and where to set it up

What	Where in XSIAM	Needed for
Marketplace KOI pack	Marketplace → search KOI → Install	Everything — it holds the integration
KOI API key	KOI console → Settings → API Access	Tests 1-7, 10
KOI integration instance	Settings → Data Sources → Add → KOI	Tests 1-7, 10
An egress IP KOI accepts	Run the instance on a Cortex engine if needed	Tests 1-7, 10
Core REST API instance	Settings → Data Sources → Add → Core REST API	Tests 8-9 only
KOI script in Action Center	Action Center → Scripts Library → upload	Tests 8-9
An endpoint group	Endpoints → tag agents, then Endpoint Groups → dynamic	Tests 8-9
"Koi Script Runner" JSON List	Settings → Object Setup → Lists	Tests 8-9

!

Two are easy to miss. The Marketplace KOI pack must be installed first — this extension ships no integration. And the Core REST API instance: without it the Script Runner's tracker stays empty and every scan run does nothing, which looks exactly like a broken playbook.

TEST 1

Connectivity and authorisation

PACK CONTENT

- KOI integration (from the Marketplace KOI pack)

TENANT SETUP

- Marketplace KOI pack installed (it holds the integration)
- A KOI API key from the KOI console

Steps

1. Settings → Data Sources → Add an instance → search KOI.
2. `Server URL: https://api.prod.koi.security/`
3. Paste the API key, click Test, then Save.
4. Open the CLI on any incident and run:
5. `!koi-inventory-list limit=5`

What to expect

✓ Test returns Success.

✓ The command returns a table of inventory items.

! A 403 here almost always means egress rather than a bad key — KOI restricts its sensitive endpoints by source IP. Run the instance on a Cortex engine whose address KOI accepts. A 401 means the key itself.

TEST 2

Event collection

PACK CONTENT

- KOI integration
- KoiContentExtension Parsing Rule

TENANT SETUP

- Test 1 passing
- Fetch events enabled on the KOI instance

Steps

1. Edit the KOI instance and tick Fetch events, then Save.
2. Wait for two collection cycles.
3. Open Query Builder and run:
4. `dataset = koi_koi_raw`
5. `| comp count() as rows by source_log_type`

What to expect

- ✓ Rows appear under Alerts, Audit, or both.
- ✓ Counts grow between cycles.
- ✓ Fields such as item_id and marketplace are populated — this pack's parsing rule does that.

! The Marketplace KOI pack ships no parsing or modeling rules. If those promoted fields are missing, this extension is not installed or has not activated yet.

TEST 3

Counting alerts correctly

PACK CONTENT

- KoiContentExtension Parsing Rule
- KOI alerts dashboard

TENANT SETUP

- Test 2 passing — events present in koi_koi_raw

Steps

1 In Query Builder run:

```
2 dataset = koi_koi_raw
3 | filter source_log_type = "Alerts"
4 | comp count() as rows, count_distinct(koi_notification_id) as
  real_alerts
```

5 Then open the KOI Alerts Dashboard and compare.

What to expect

- ✓ rows is far larger than real_alerts.
- ✓ Dashboard alert counts match real_alerts, not rows.
- ✓ Counts stay stable as fetches repeat.

! rows greatly exceeding real_alerts is CORRECT — the Marketplace integration re-sends every still-open alert on each fetch. Only koi_notification_id identifies an alert. Every widget here de-duplicates on it first.

TEST 4

Alert triage

PACK CONTENT

- KOI Ext IR - Alert Triage
- KOI Ext IR - Extract Alert Context
- 19 KOI incident fields + incident type

TENANT SETUP

- Test 1 passing
- A KOI alert in XSIAM

Steps

1. Open a KOI alert.
2. Attach the triage playbook, or run:
3. `!setPlaybook playbookId="KOI Ext IR - Alert Triage"`
4. Watch the Work Plan, then read the War Room.

What to expect

- ✓ Context is extracted from the alert payload.
- ✓ A verdict is posted: Malicious, Suspicious or Benign.
- ✓ Low risk auto-closes; critical and high escalate.
- ✓ The 19 KOI alert fields are written onto the alert.

! If everything comes out Suspicious, the alert carried no KOI detail for the playbook to read. Check that your correlation rule passes the KOI fields through.

TEST 5

Item and device investigation

PACK CONTENT

- KOI Ext IR - Investigate Item
- KOI Ext IR - Investigate Device
- KOI Ext IR - Enrich Item

TENANT SETUP

- Test 1 passing
- An item id and its marketplace, and a hostname
- Nothing else — the XQL engine XSIAM uses for execution evidence is built in

Steps

1. Automation → Playbooks → KOI Ext IR - Investigate Item → Run.
2. Supply item_id and marketplace.
3. Repeat with KOI Ext IR - Investigate Device and a hostname.

What to expect

- ✓ An investigation summary with organisation exposure.
- ✓ Both governance counts — on blocklist AND on allowlist.
- ✓ The device view lists what KOI inventoried on that host.
- ✓ With Cortex XDR present, execution evidence is added.

! This pack builds its device view from inventory filtered by host plus Cortex XDR data. KOI's own device record — last seen, registration, logged-on user — needs the custom integration's device commands and is not available here.

TEST 6

Gated response

PACK CONTENT

- KOI Ext IR - Block and Remediate
- KOI Ext IR - Investigate Item

TENANT SETUP

- Test 1 passing
- An item id and marketplace
- Permission to approve a task

Steps

1. Automation → Playbooks → KOI Ext IR - Block and Remediate → Run.
2. Supply item_id and marketplace.
3. Open the pending approval task and read it.
4. Approve, or leave it pending.

What to expect

- ✓ The run PARKS on an approval task and waits.
- ✓ The approval shows the investigation and governance state.
- ✓ Nothing reaches the blocklist until a human approves.
- ✓ An item already blocked short-circuits.

! Re-run this after any change to the response playbook. If a run ever completes without stopping for approval, stop and investigate before using the pack in production.

TEST 7

Proactive hunting

PACK CONTENT

- KOI Ext Hunting - MCP Server Audit
- KOI Ext Hunting - Hunt Sweep
- KOI Ext Hunting - Hunt Match Investigation

TENANT SETUP

- Test 1 passing
- Nothing else — the hunt sweep queries xdr_data, which XSIAM holds natively

Steps

1. Automation → Playbooks → KOI Ext Hunting - MCP Server Audit → Run.
2. Then run KOI Ext Hunting - Hunt Sweep.
3. Optionally set hunt_set and min_risk.
4. Read the War Room match table.

What to expect

- ✓ The audit lists MCP servers at or above the threshold.
- ✓ The sweep runs its hunts and investigates what it finds.
- ✓ Block candidates are routed to an analyst gate, never blocked automatically.

! The sweep checks the XQL engine is answering before it runs, and stops cleanly if not. On XSIAM the engine is provisioned by the platform, so that check should simply pass.

TEST 8

Script Runner — refresh the tracker

PACK CONTENT

- KOI Ext Script Runner - Refresh Job
- KOI Ext Script Runner - Refresh Entry
- KoiScanTracker (our automation — not the KOI script)
- List: Koi Script Runner

TENANT SETUP

- Core REST API integration instance configured ← the one people miss
- The KOI deployment script — download it from YOUR KOI console, then upload it to Action Center → Scripts Library. This is KOI's script, not a Cortex one. It must take no parameters
- An endpoint group — tag the agents, then use a dynamic group
- The Koi Script Runner List created (step 1)

Steps

1. Settings → Object Setup → Lists → New List, type JSON, named exactly Koi Script Runner.
2. Open Lists/README.md in the pack and copy the JSON block — it is formatted ready to paste (do NOT copy from list-Koi_Script_Runner.json; its data field is escaped).
3. Edit two things only: endpoint_groups (your group) and script.name (the KOI script exactly as it appears in Action Center). One entry per OS.
4. Everything else — tracker_list, rescan_interval_hours, max_endpoints — already has working defaults.

What to expect

- ✓ A tracker List appears under Lists, named as in your entry.
- ✓ It fills with one row per endpoint: an id and a last-scan value.
- ✓ Re-running adds new endpoints without disturbing existing rows.

! Without a Core REST API instance this test cannot pass — the refresh reads endpoint groups through it, and the tracker simply stays empty.

TEST 9

Script Runner — scan due endpoints

PACK CONTENT

- KOI Ext Script Runner - Scan Job
- KOI Ext Script Runner - Process Config Entry
- KOI Ext Script Runner - Execute Endpoint Script
- KoiScanTracker (our automation)

TENANT SETUP

- Test 8 passing — the tracker List has rows
- Connected agents in the group, matching the entry's endpoint_os

Steps

1. Automation → Jobs → New Job → time-triggered → playbook KOI Ext Script Runner - Scan Job.
2. Enable the Job, then Run now.
3. Open the run, then check Action Center.
4. Run it a second time immediately.

What to expect

- ✓ The script is dispatched to due, connected endpoints.
- ✓ Those endpoints get a last-scan timestamp in the tracker.
- ✓ Offline and wrong-OS endpoints stay due and retry later.
- ✓ The second run does nothing — all inside the rescan interval.

! SKIPPED means nothing is due right now and sends no email, by design. STILL RUNNING means the script was dispatched and Action Center is finishing on its own. Neither is a failure.

TEST 10

Alert fields and layout

PACK CONTENT

- 19 KOI incident fields
- KOI Supply Chain Alert incident type
- its layout

TENANT SETUP

- Test 4 passing — triage has run on an alert
- The incident type and layout installed with this pack

Steps

1. Open an alert that triage has processed.
2. Check its type is KOI Supply Chain Alert.
3. Read the KOI Alert tab.

What to expect

- ✓ 19 KOI fields are populated on the alert itself.
- ✓ They are grouped: item, risk and verdict, affected host, governance, summary.
- ✓ An analyst sees the picture without opening the War Room.

! This is what the extension adds over the Marketplace pack alone. If fields are empty, triage has not run on that alert yet — it is triage that writes them.

IF SOMETHING FAILS

Check the prerequisite before the product

Integration not offered	Marketplace KOI pack not installed	Install it first — this extension ships no integration
403 on KOI commands	Source IP not accepted by KOI	Run the instance on a Cortex engine KOI accepts
Promoted fields missing	This extension not installed or not active	Re-check the pack installed and events arrived after it
Everything triages Suspicious	The alert carried no KOI detail	Check the correlation rule passes KOI fields through
Alert fields empty	Triage has not run on that alert	Triage writes them — run it first
Hunt sweep skips immediately	The XQL engine did not answer	Re-run. XSIAM provisions the engine, so this should not persist
Tracker List stays empty	No Core REST API instance	Configure it — Test 8 cannot pass without it
A Job never fires	It was created disabled	Enable it and confirm a next-run time is shown

Seven of these eight are tenant setup, not the pack. Check the amber panel on the test before raising a defect.

SIGN-OFF

One line of evidence per test

1. Test returns Success; inventory returns rows
2. koi_koi_raw grows; promoted fields are populated
3. Distinct alerts far below raw rows; dashboard matches
4. A verdict is reached; low risk auto-closes
5. Investigation shows exposure and both governance counts
6. Response parks on approval; blocklist untouched
7. MCP audit returns servers; hunt sweep completes
8. Tracker List is created and fills with endpoints
9. Scan marks only due, connected endpoints
10. 19 KOI fields populated on the alert layout

All ten green — the extension is ready for production use.

Considering the custom KOI integration instead? See the deployment comparison for what differs.